

Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente

Licenciatura en Tecnología Digital — Universidad Torcuato Di Tella

Información del curso

- **Duración:** 16 semanas (semestral).
- **Encuentros:** un encuentro semanal (martes), en dos bloques de 95 min (B1 y B2).
- **Modalidad:** Teórico-práctica, con formato de taller.
- **Correlatividad:** No es correlativa de TD VI (Inteligencia Artificial). No se asume que todos los estudiantes hayan visto previamente fundamentos de redes neuronales o Transformers.

Descripción del curso

La materia ofrece una formación avanzada en los fundamentos y la ingeniería de sistemas basados en modelos de lenguaje de gran escala. El recorrido comienza con los principios matemáticos de redes neuronales y el estudio profundo de Transformers y mecanismos de atención, continúa con la representación del texto (tokenización y embeddings) y con los procesos de entrenamiento y alineamiento de LLMs, y avanza hacia técnicas modernas de interacción y control. Con un enfoque fuertemente práctico y de reconstrucción desde cero —*construir, no postular*—, los estudiantes diseñarán e implementarán arquitecturas que integran recuperación de información, memoria, planificación y uso de herramientas externas, culminando en la construcción de Sistemas Multi Agente (Agentic AI). El objetivo es formar profesionales capaces de comprender en profundidad estas tecnologías y, al mismo tiempo, desarrollar soluciones de inteligencia artificial robustas aplicables a problemas reales.

Objetivos de aprendizaje

Al finalizar el curso, los estudiantes podrán:

- Comprender los principios y la arquitectura detrás de los LLMs y Transformers, sin necesidad de un trasfondo matemático profundo.
- Explicar la representación del texto en LLMs: tokenización (BPE y variantes) y embeddings, y su impacto en el comportamiento del modelo.
- Analizar críticamente los procesos de preentrenamiento, fine-tuning y alineamiento de modelos (RLHF, DPO, Constitutional AI).
- Diseñar pipelines de RAG, agentes con memoria y planificación multi-paso.
- Implementar sistemas multi-agente robustos aplicables a problemas del mundo real.
- Evaluar sesgos, limitaciones éticas y riesgos de los sistemas generativos y agénticos.

Herramientas y stack del curso

- **Tooling libre y open-source.** Todas las herramientas y modelos usados en clase y en los proyectos son gratuitos y de código abierto. No se depende de APIs pagas.
- **Runtime de LLM:** Ollama (local) con modelos abiertos; HuggingFace **transformers** para carga de modelos y tokenizers. Notebooks en Jupyter / Google Colab.

- **Frameworks:** HuggingFace, LangChain, LangGraph, LlamaIndex, AutoGen, CrewAI.
- **Producción de código con asistencia de IA.** En línea con el objeto de la materia, se asume y se alienta el uso de asistentes de IA para escribir código. La evaluación no mide la capacidad de teclear código, sino la **comprensión conceptual**: por qué cada pieza está, qué problema concreto resuelve y cómo se articula con el resto. Los proyectos se juzgan por las decisiones de arquitectura y el entendimiento demostrado; el examen final verifica conceptos, no implementación.

Contenido por unidades

Unidad 1 — Fundamentos: de vectores a los embeddings

Semanas 1–4

Semana	Tema	Bloque 1	Bloque 2
1	Fundamentos matemáticos del aprendizaje profundo	Vectores, producto interno, capa afín, no linealidad; pérdida y gradiente	Backprop como diferenciación automática. Lab: motor de autograd desde cero (micrograd)
2	Transformer desde cero	De un bigrama a self-attention: promedio del pasado → máscara → softmax → atención (q·k)	Multi-head, bloque Transformer; apéndice encoder-decoder (BERT / GPT / T5). Lab: notebook estilo nanoGPT
3	Tokenización	Unicode → bytes UTF-8 → BPE (merges) → encode/decode	Regex splitting (GPT-2/4), tokens especiales, tokenizers reales (tiktoken · SentencePiece · WordPiece · Unigram). Lab: tokenizer desde cero + HF <i>AutoTokenizer</i>
4	Embeddings: de token a vector	one-hot → word2vec / GloVe → similitud coseno	Búsqueda semántica (puente a RAG). Lab: explorar y comparar embeddings

Nota sobre PyTorch (opcional). PyTorch no se dicta formalmente. El lab de la Semana 1 construye el mecanismo de autograd desde cero (micrograd) y las notebooks siguientes usan PyTorch en contexto. Para quien quiera formalizar el framework, se recomienda el tutorial oficial *Learn the Basics* (pytorch.org/tutorials): es la versión industrial de exactamente lo que se construyó a mano.

Unidad 2 — Entrenamiento, fine-tuning y alineamiento de LLMs

Semanas 5–6

Semana	Tema	Bloque 1	Bloque 2
5	Preentrenamiento y fine-tuning eficiente	Datos, curación, scaling laws, causalidad del lenguaje	Instruction tuning, PEFT, LoRA, QLoRA. Lab: fine-tuning con HuggingFace PEFT sobre un modelo abierto chico
6	Alineamiento y evaluación	RLHF, DPO, Constitutional AI	Benchmarks (MMLU, HellaSwag), hallucinations, sesgos, responsible AI

Semana	Tema	Bloque 1	Bloque 2
		(conceptual); reward models	

Unidad 3 — Ingeniería de prompts e interacción avanzada

Semana 7

Semana	Tema	Bloque 1	Bloque 2
7	Prompt engineering y structured outputs	Zero/few-shot, Chain-of-Thought, Tree-of-Thought, Self-Consistency	Structured outputs (JSON mode), function calling / tool use. Lab: llamadas a herramientas con Ollama + HuggingFace (modelos abiertos)

Unidad 4 — RAG, memoria y contexto extendido

Semanas 8–10

Semana	Tema	Bloque 1	Bloque 2
8	Retrieval-Augmented Generation (RAG)	Naive RAG: chunking, embeddings, vector stores (FAISS, Chroma); RAG desde cero recuperando por similitud coseno (enlaza con Semana 4)	RAG avanzado: reranking, HyDE, parent-child chunking, evaluación con RAGAS. Lab: LlamaIndex
9	Memoria y gestión del contexto	Memoria en sistemas LLM: short/long-term, episodic, scratchpad, external memory stores	Context window, compression, summarization chains
10	Graph RAG y pipeline completo	Graph RAG y Knowledge Graphs: recuperación sobre grafos estructurados (Neo4j + LLMs)	Lab: pipeline RAG end-to-end con memoria

Unidad 5 — Agentes LLM: herramientas y planificación

Semanas 11–12

Semana	Tema	Bloque 1	Bloque 2
11	Arquitectura de agentes y uso de herramientas	Componentes del agente (perfil, memoria, planificación, acción); ReAct ("Synergizing Reasoning and Acting")	Herramientas externas (web search, code execution, Model Context Protocol). Lab: del agente hecho a mano (notebook DIY sobre Ollama) a su versión en LangChain
12	Planificación multi-paso y robustez	Plan-and-execute, ReWOO, reflexión y autocorrección, memoria episódica	Evaluación y robustez de agentes (benchmarks agénticos, failure modes, trazabilidad)

Unidad 6 — Sistemas Multi Agente (Agentic AI)

Semanas 13–16

Semana	Tema	Bloque 1	Bloque 2
13	Introducción a MAS y arranque de LangGraph	Motivación, taxonomía y paradigmas de diseño: grafo con estado (LangGraph), conversacional (AutoGen), roles y tareas (CrewAI). Patrones de comunicación: broadcast, peer-to-peer, jerarquía	Fundamentos de LangGraph: estado, nodos, aristas, ciclos. Lab: primer sistema multi-agente en LangGraph
14	LangGraph en profundidad	Flujos con estado, ciclos y orquestación en LangGraph	Patrones avanzados como grafos: supervisor-worker, debate entre agentes, critic pattern, agentes especializados
15	Coordinación, seguridad y LangGraph aplicado	Coordinación, confianza y seguridad: conflictos entre agentes, prompt injection en MAS, human-in-the-loop, guardrails	Lab: human-in-the-loop (interrupts) y guardrails en LangGraph; trabajo sobre el proyecto final
16	Consultas y cierre	Consultas del TP y seguimiento del proyecto final	Cierre de la materia: reflexión sobre el estado del arte y perspectivas futuras

Cronograma de trabajos prácticos y evaluaciones

Trabajo práctico / evaluación	Instancia	Fecha
Proyecto 1	Enunciado	domingo 6/9/2026
	Entrega	domingo 27/9/2026
Proyecto 2	Enunciado	domingo 27/9/2026
	Entrega	domingo 18/10/2026
Proyecto final	Enunciado	domingo 18/10/2026
	Entrega	domingo 29/11/2026
	Presentación	Cierre de cursada
Examen	Final	Cierre de cursada

Metodología de trabajo en clase

Clases teóricas

- Presentación de fundamentos: exposición de los principios de cada unidad, con énfasis conceptual antes que en derivaciones matemáticas exhaustivas.

- Reconstrucción desde cero: cada pieza nueva aparece porque la anterior se queda corta (*construir, no postular*); nada del stack "aparece de la nada".
- Discusión y resolución de dudas, y lectura de papers seminales (Vaswani et al., Brown et al., Ouyang et al., entre otros).

Clases prácticas

- Guías semanales de laboratorio con implementación en Python, sobre tooling libre y open-source (Ollama, HuggingFace).
- Tutoriales sobre frameworks relevantes (HuggingFace, LangChain, LangGraph, LlamaIndex).
- Proyectos grupales (en grupos de hasta 4 integrantes), con foco en arquitecturas reales de fine-tuning, RAG, agentes y sistemas multi-agente. El código puede producirse con asistencia de IA; lo que se evalúa es la comprensión conceptual y las decisiones de diseño.

Forma de evaluación y requisitos de aprobación

La materia se evalúa mediante **tres proyectos de implementación** en computadora (con entrega y sin recuperatorio, que podrán resolverse en grupos de hasta 4 integrantes) y **un examen final individual de carácter aprobado/desaprobado**.

- La **nota numérica** de la materia proviene íntegramente de los tres proyectos.
- El **examen final** es un requisito de aprobación de tipo aprobado/desaprobado, **en fecha de examen aparte** (fuera del cronograma de clases): no pondera en la nota numérica, pero es necesario aprobarlo para aprobar la materia. Verifica la comprensión conceptual integradora del curso.

Para aprobar la materia se debe: (a) obtener una nota mayor o igual a 60 (sobre 100) en **cada uno** de los tres proyectos, y (b) **aprobar el examen final**.

Reprueba la materia quien no apruebe alguno de los tres proyectos, o quien no apruebe el examen final. El examen final admite una única instancia de recuperación.

Ponderación de la nota final

El 100% de la nota numérica proviene de los tres proyectos. La forma de ponderarlos entre sí está por definir.

Escala de nota final

Se aplica sobre la nota numérica de la materia (los tres proyectos).

Promedio de proyectos	Nota final
[90, 100]	A
[85, 90)	A-
[80, 85)	B+
[75, 80)	B
[70, 75)	B-
[65, 70)	C+

Promedio de proyectos	Nota final
-----------------------	------------

[60, 65)	C
----------	---

Requisitos previos

- Programación avanzada: Python fluido, estructuras de datos, programación orientada a objetos.
- Álgebra lineal y probabilidad: vectores, matrices, distribuciones (nivel introductorio).
- Machine learning introductorio: conceptos básicos de entrenamiento, función de loss, gradiente.
- Uso de notebooks y entornos de ejecución (local y en la nube).

Bibliografía

La bibliografía está sujeta a actualizaciones durante la cursada.

Principal

- Vaswani, A. et al. (2017). *Attention Is All You Need*. arXiv:1706.03762.
- Brown, T. et al. (2020). *Language Models are Few-Shot Learners* (GPT-3). NeurIPS 2020.
- Ouyang, L. et al. (2022). *Training language models to follow instructions with human feedback* (InstructGPT / RLHF).
- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.
- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*.
- Park, J. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*.

Complementaria / avanzada

- Karpathy, A. — *Let's build the GPT Tokenizer* (2024) y repositorio [minbpe](#); *nanoGPT* y *micrograd*. Base de la reconstrucción desde cero de las Unidades 1.
- PyTorch — *Learn the Basics / Deep Learning with PyTorch: A 60 Minute Blitz* ([pytorch.org/tutorials](#)). Introducción **opcional** al framework para quienes quieran profundizar en la implementación.
- Anthropic (2023–2025). Model Card series y documentación de Claude.
- Documentación oficial de HuggingFace, LangChain, LangGraph, LlamaIndex, AutoGen, CrewAI y Ollama.
- Se proporcionarán capítulos específicos de libros o artículos académicos al final de cada clase para profundizar en los temas tratados.